ESIEE-IT

Projet IA — Utilisation de l'IA

GridLock

Application de suivi des participations étudiantes
aux JPO, Salons et Forums

Document Technique — Développement & Qualité

US22 · US23 · US24 · US25 · US26 · US27 · US28 · US29

Avril 2026

US22 — Initialiser le socle applicatif du projet

22.1 Objectif

L'US22 pose le socle de départ du projet GridLock : une structure de dossiers cohérente, les outils de qualité configurés et un premier test exécutable. L'objectif est de fournir à l'équipe un point d'entrée propre, reproductible et versionnable avant tout développement fonctionnel.

22.2 Structure des dossiers

Chemin	Description
gridlock/	Racine du projet
├── **frontend/**	Application React + Vite + Tailwind
│ ├── src/	Code source React
│ │ ├── components/	Composants réutilisables (Button, Badge, Toast…)
│ │ ├── pages/	Pages : Dashboard, Events, Students, Login
│ │ ├── hooks/	Hooks personnalisés (useAuth, useEvents, useStudents)
│ │ ├── services/	Appels API centralisés (api.js, auth.service.js…)
│ │ ├── context/	AuthContext, état global
│ │ └── utils/	Fonctions utilitaires (formatDate, statusColor…)
│ ├── .env.local	Variables d'environnement frontend (VITE_API_URL)
│ └── vite.config.js	Configuration Vite
├── **backend/**	API REST Node.js + Express
│ ├── src/	Code source backend
│ │ ├── routes/	Définition des routes (events, students, auth…)
│ │ ├── controllers/	Logique métier par ressource
│ │ ├── middleware/	auth.middleware.js, role.middleware.js, validate.js
│ │ ├── validators/	Schémas Zod par route
│ │ └── utils/	jwt.utils.js, hash.utils.js
│ ├── prisma/	Schéma Prisma et migrations
│ │ ├── schema.prisma	Modèle de données (User, Student, Event, Participation)
│ │ └── seed.js	Script de peuplement initial (données de démo)
│ └── .env	Variables d'environnement backend (DATABASE_URL, JWT_SECRET)
├── **docker-compose.yml**	Orchestration locale : frontend, backend, PostgreSQL
├── **.gitignore**	Exclusions Git (node_modules, .env, dist)
└── **README.md**	Documentation principale du projet

22.3 Configuration initiale

ESLint + Prettier

Les fichiers .eslintrc.json et .prettierrc sont présents à la racine de chaque sous-projet (frontend et backend). Les règles appliquées d'entrée de jeu sont :

- Indentation : 2 espaces
- Guillemets simples
- Point-virgule obligatoire
- prefer-const activé
- no-unused-vars activé
- no-console activé en production (warn en dev)

Vérification initiale

Le socle est considéré livré lorsque les commandes suivantes s'exécutent sans erreur :

```
cd frontend && npm install && npm run lint && npm run build
cd backend && npm install && npm run lint && npx prisma generate
docker-compose up --build  # Tous les containers démarrent
```

22.4 Commit de livraison

L'initialisation du socle est livrée dans un commit atomique dédié, conformément aux conventions US17 :

```
chore(init): scaffold project structure with frontend, backend, Prisma and Docker
```

Ce commit inclut : structure dossiers, .gitignore, .env.example (sans secrets), docker-compose.yml, configuration ESLint/Prettier, schéma Prisma initial, premier test Jest/Vitest vide exécutable.

US23 — Implémenter le parcours utilisateur principal

23.1 Objectif

Le parcours principal de GridLock est celui de Sophie, Coordinatrice Sourcing, qui inscrit un étudiant à un événement. Cette US couvre l'implémentation complète de bout en bout de ce parcours : de la connexion à la confirmation visuelle de l'inscription.

23.2 Étapes du parcours implémenté

#	Étape	Route	Résultat
1	**Login admin**	POST /api/auth/login	Retourne JWT stocké en localStorage
2	**Créer un événement**	POST /api/events	Validation Zod + 201 Created
3	**Rechercher un étudiant**	GET /api/students?q=VIL	Auto-complétion, min 2 chars
4	**Inscrire l'étudiant**	POST /api/events/:id/participations	Vérification doublon, 201 ou 409
5	**Afficher les inscrits**	GET /api/events/:id/participations	Liste avec student + statut
6	**Changer le statut**	PUT /api/participations/:id	Confirmé → Présent ou Absent
7	**Tableau de bord**	GET /api/dashboard	KPIs globaux, admin/director only

23.3 Critères de réalisation

Frontend

- Page /login fonctionnelle avec validation côté client et retour d'erreur 401 affiché
- Liste des événements filtrables (mois, type) chargée depuis GET /api/events
- Fiche événement avec tableau des participants et bouton 'Ajouter un participant'
- Modale d'inscription avec auto-complétion sur GET /api/students?q=
- Toast de succès (vert) ou d'erreur (rouge) après soumission
- Redirect automatique vers /login si 401 reçu (intercepteur Axios)

Backend

- Middleware auth.middleware.js vérifiant le JWT sur toutes les routes protégées
- Middleware role.middleware.js autorisant uniquement les admins pour POST /participations
- Validation Zod sur le body de POST /participations : studentId (uuid requis), statut (enum requis)
- Vérification doublon via SELECT COUNT(*) avant INSERT
- Retour 409 avec corps {error: 'Etudiant déjà inscrit à cet événement'} si doublon
- Retour 201 avec l'objet participation créé si succès

23.4 Test associé

Un test d'intégration couvre le chemin nominal et le cas d'erreur 409 :

```
POST /api/events/uuid/participations — cas nominal → 201 Created
POST /api/events/uuid/participations — doublon → 409 Conflict
POST /api/events/uuid/participations — sans JWT → 401 Unauthorized
```

23.5 Découpage en commits

1. feat(auth): implement JWT login endpoint and middleware
2. feat(events): implement GET /events list and POST /events creation
3. feat(students): implement GET /students autocomplete endpoint
4. feat(participation): implement POST /participations with duplicate check
5. feat(frontend): implement login page with JWT storage and axios interceptor
6. feat(frontend): implement event list with filters and event detail view
7. feat(frontend): implement add participant modal with autocomplete
8. test(participation): add integration tests for nominal and error cases

US24 — Implémenter les fonctionnalités cœur du MVP

24.1 Objectif

Au-delà du parcours principal, le MVP GridLock comprend un ensemble de fonctionnalités complémentaires qui constituent la valeur réelle du produit : gestion complète des événements, des statuts, des vues récapitulatives et du tableau de bord. Chaque fonctionnalité est reliée à une US ou à un scénario personas.

24.2 Inventaire des fonctionnalités cœur

Fonctionnalité	US source	Implémentation	MVP
CRUD Événements	US04	Création, modification, suppression d'événements avec validation Zod côté backend	✓
Inscription participant	US03 / US11	Modale d'ajout, auto-complétion, vérification doublon (409)	✓
Gestion des statuts	US03	Dropdown inline Confirmé / Présent / Absent sur la fiche événement	✓
Badge ambassadeur	US03	Checkbox par participant, persistée en base	✓
Récap par événement	US03	Compteurs : inscrits, présents, ambassadeurs	✓
Récap par étudiant	US02	Vue individuelle avec filtre par type, lecture seule pour étudiant	✓
Filtres événements	US04	Filtre mois, type, formation sur la liste	✓
Dashboard KPI	US02	Total événements, participations, présents, ambassadeurs — admin/director	✓
Auth JWT	US04	Login, token expiration 24h, redirect /login sur 401	✓
Auto-complétion étudiants	US03	GET /api/students?q= avec min 2 chars, suggestions	✓

24.3 Règles métier appliquées

Événements

- Le champ 'type' est une liste fermée : JPO, Salon, Forum, Évènement — validée par Zod côté backend
- La suppression d'un événement entraîne la suppression en cascade de toutes ses participations (ON DELETE CASCADE en BDD)
- Un événement ne peut être créé ou modifié que par un utilisateur admin

Participations

- Un étudiant ne peut être inscrit qu'une fois par événement — contrainte UNIQUE (event_id, student_id) en BDD
- Le statut par défaut à l'inscription est 'confirmé'
- Le passage à 'présent' ou 'absent' est effectué post-événement par l'admin
- La suppression d'un participant nécessite une confirmation modale avant exécution

Rôles

- Les actions d'écriture (POST, PUT, DELETE) sont toutes protégées par role.middleware.js

- Le Director peut lire toutes les données mais ne peut rien modifier
- L'étudiant ne peut accéder qu'à ses propres participations via GET /students/me/participations

24.4 Dépendances entre fonctionnalités

L'ordre de développement recommandé respecte les dépendances suivantes :

9. Socle auth (US22) — prérequis de tout le reste
10. CRUD Événements — prérequis de la gestion des participants
11. Auto-complétion étudiants — prérequis de la modale d'inscription
12. Inscription + statuts — cœur du MVP (US23)
13. Tableau de bord KPI — dernier, agrège tout
14. Vue étudiant (lecture seule) — optionnelle selon avancement

US25 — Réaliser une interface cohérente et accessible

25.1 Objectif

L'interface GridLock suit le mini guide UI défini en US08. Cette US documente les choix d'implémentation CSS, les vérifications d'accessibilité réalisées et les limites connues du MVP sur ce périmètre.

25.2 Alignement avec le guide UI (US08)

Palette et typographie

- Les variables CSS Tailwind sont configurées dans tailwind.config.js pour exposer les couleurs du guide (navy: '#0B1F3A', accent: '#1A56A8', gold: '#F59E0B'…)
- Les polices Syne (titres) et Inter (corps) sont chargées via Google Fonts dans index.html

Composants récurrents implémentés

- Button.jsx : variantes primary, secondary, danger — props variant, disabled, loading (spinner)
- Badge.jsx : pills de statut colorées — variantes confirmé (gold), présent (vert), absent (gris)
- Toast.jsx : notification bas-droite, auto-dismiss 3s, variantes success / error
- Modal.jsx : overlay + focus trap + fermeture Echap et clic hors modale
- Sidebar.jsx : fixe 240px desktop, overlay hamburger mobile, item actif avec bordure gauche bleue
- EmptyState.jsx : composant réutilisable avec icône, message et CTA optionnel

25.3 Vérifications d'accessibilité

Critère	Mécanisme	Périmètre	Statut
Navigation clavier	Tab / Shift+Tab	Tous les formulaires, modales et tableaux du parcours principal	✓ Vérifié
Focus visible	outline ring 2px #1A56A8	Tous les inputs, boutons et liens	✓ Vérifié
Contraste couleurs	Ratio ≥ 4.5:1	Textes sur fond Navy ou blanc — conforme WCAG AA	✓ Vérifié
Labels de formulaire	<label> explicite	Tous les champs obligatoires	✓ Vérifié
Messages d'erreur	aria-live='polite'	Validation en temps réel — annoncée aux lecteurs d'écran	Partiel
Responsive mobile	Breakpoints Tailwind	md: sidebar en overlay, sm: tableaux scrollables horizontalement	✓ Vérifié
Alt texte icônes	aria-label	Boutons icônes (suppression, édition) avec label explicite	✓ Vérifié

25.4 Limites connues

- Les tableaux de données ne sont pas encore optimisés pour les lecteurs d'écran (pas de scope/headers ARIA complets) — à adresser en V2
- Le mode sombre n'est pas implémenté dans le MVP
- Aucun audit Lighthouse automatique n'est intégré en CI pour le MVP — à prévoir avec US31

US26 — Gérer les états vides, erreurs et cas limites

26.1 Objectif

Un produit crédible ne casse pas dès qu'on sort du scénario idéal. Cette US documente l'ensemble des cas limites identifiés, leur traitement dans le MVP et les limites connues.

26.2 États vides — règle générale

Chaque liste ou tableau de l'application dispose d'un état vide explicite via le composant EmptyState.jsx. Les états vides définis sont :

- Liste d'événements vide : 'Aucun événement cette année' + bouton 'Créer le premier événement' (admin uniquement)
- Liste de participants vide : 'Aucun participant inscrit à cet événement' + bouton 'Inscrire un étudiant'
- Vue récap étudiant sans participation : 'Vous n'avez pas encore participé à un événement.'
- Recherche auto-complétion sans résultat : 'Aucun étudiant trouvé pour "[terme]"'

26.3 Taxonomie des erreurs et traitements

Cas limite	Type	Traitement	Statut
Étudiant déjà inscrit	Métier	409 Conflict → toast rouge 'Cet étudiant est déjà inscrit'	✓ Traité
Token JWT expiré	Technique	401 → intercepteur Axios → redirect /login + message 'Session expirée'	✓ Traité
Champ obligatoire vide	Validation	Bouton submit désactivé + message sous le champ concerné	✓ Traité
Aucun résultat auto-complétion	UX	Message 'Aucun étudiant trouvé' dans la liste déroulante	✓ Traité
Suppression d'un événement avec participants	Métier	DELETE CASCADE en BDD — avertissement de confirmation modale avant action	✓ Traité
Rôle insuffisant (403)	Autorisation	Page 403 avec message explicite + bouton retour	✓ Traité
Route inexistante (404)	Technique	Page 404 avec bouton retour dashboard	✓ Traité
Erreur serveur (500)	Technique	Banner rouge générique + log serveur (pas de stack trace client)	✓ Traité
Étudiant non trouvé dans la base	Métier	Message 'Aucun étudiant correspondant' — pas de saisie libre pour le MVP	Limite MVP
Connexion BDD perdue	Technique	Prisma renvoie 500 — message générique affiché, service relancé via Docker	Limite MVP
Doublon d'événement (même date + structure)	Métier	Aucune contrainte DB — à la charge de l'admin pour le MVP	Hors MVP

26.4 Stratégie de gestion des erreurs côté backend

Middleware global d'erreurs (Express)

Un middleware errorHandler.js est déclaré en dernier dans app.js. Il intercepte toutes les erreurs non gérées et retourne une réponse normalisée :

```
{ error: 'Une erreur est survenue. Veuillez réessayer.' }  // 500
{ error: message }  // 400, 401, 403, 404, 409 — message métier explicite
```

La stack trace n'est jamais envoyée au client. Elle est uniquement loggée côté serveur (console.error en dev, fichier de log en prod).

Intercepteur Axios côté frontend

Un intercepteur de réponse Axios est configuré dans services/api.js. Il gère :

- 401 → déconnexion + redirect /login + toast 'Session expirée'
- 403 → redirect /403
- 500 → toast rouge 'Erreur serveur — veuillez réessayer'
- Erreurs réseau (offline) → toast rouge 'Connexion impossible'

US27 — Préparer les données de démonstration et de test

27.1 Objectif

Un jeu de données crédible, cohérent avec le domaine éducatif d'ESIEE-IT, est nécessaire pour la démonstration au jury et pour les tests automatisés. Ces données ne contiennent aucune donnée personnelle réelle.

27.2 Comptes utilisateurs de démo

Email	Mot de passe	Rôle	Usage en démo
sophie@esiee-it.fr	Admin123!	**admin**	Coordinatrice Sourcing
marc@esiee-it.fr	Director1!	**director**	Directeur Admissions
enzo.martin@edu.esiee-it.fr	Student1!	**student**	Étudiant B3 Coding Cergy

27.3 Événements de démo

Mois	Type	Date	Structure	Événement	Ville
Octobre	**JPO**	14/10/2025	ESIEE-IT Pontoise	JPO Automne Pontoise	Pontoise
Novembre	**Forum**	22/11/2025	Lycée Camille Pissarro	Forum Orientation	Pontoise
Janvier	**Salon**	18/01/2026	Paris Expo Porte de Versailles	Salon Studyrama	Paris 15e
Mars	**JPO**	07/03/2026	ESIEE-IT Cergy	JPO Printemps Cergy	Cergy
Avril	**Salon**	12/04/2026	Parc Floral	Salon de l'Étudiant	Paris 12e

27.4 Participations de démo

Les étudiants de démo couvrent les cas suivants :

- Enzo MARTIN (Coding Cergy B3) : présent à JPO Automne Pontoise (statut Présent, ambassadeur), confirmé pour Forum Orientation, absent au Salon Studyrama
- Léa DUPONT (DSNS B2) : confirmée pour JPO Printemps Cergy, aucune autre participation → valide l'état 'peu de participations'
- Maxence VILLARD (Cybersécurité B3) : présent à 4 événements → valide l'affichage d'un historique long et les compteurs KPI
- Yasmine OUALI (Marketing B1) : aucune participation → valide l'état vide de la vue étudiant

27.5 Chargement des données

Script de seed Prisma

```
cd backend && npx prisma db seed
```

Le script prisma/seed.js est idempotent : il vérifie l'existence des données avant d'insérer (upsert). Il peut être relancé sans créer de doublons.

Réinitialisation complète

```
npx prisma migrate reset --force  # Recrée la BDD et relance le seed
```

27.6 Données dans les tests

Les tests unitaires et d'intégration n'utilisent pas les données de seed. Ils utilisent des factories dédiées (fichiers test/factories/) qui créent des objets en mémoire ou des entrées BDD isolées dans un schéma de test séparé.

- Base de test : DATABASE_URL_TEST dans .env.test
- Nettoyage entre tests : beforeEach(() => prisma.$executeRaw('TRUNCATE ...')
- Les données de seed restent exclusivement pour la démo et le développement local

US28 — Définir les règles de qualité de code et de commentaires

28.1 Objectif

Ce document établit les règles de qualité de code applicables à GridLock. Ces règles s'appliquent à tout le code produit — par les développeurs humains comme par les agents IA — et sont vérifiables via ESLint, Prettier et SonarQube.

28.2 Règles de qualité

Axe	Règle	Description	Vérification
Lisibilité	**Longueur de fonction**	Max 30 lignes par fonction/méthode	ESLint max-lines-per-function
Lisibilité	**Nommage**	camelCase JS, UPPER_SNAKE pour constantes, PascalCase composants React	ESLint naming-convention
Duplication	**DRY**	Pas de bloc dupliqué > 5 lignes — extraire en utilitaire ou hook	SonarQube duplications
Complexité	**Cyclomatique**	Complexité < 10 par fonction	SonarQube cognitive complexity
Séparation	**Single Responsibility**	1 fichier = 1 responsabilité. Pas de logique métier dans un composant React	Revue de code
Commentaires	**JSDoc utiles**	Commenter les fonctions non triviales (règles métier, workaround)	Revue de code
Commentaires	**Interdits**	Pas de commentaires 'TODO sans ticket', pas de code commenté versionné	ESLint no-warning-comments
Formatage	**Prettier**	Indentation 2 espaces, guillemets simples, point-virgule obligatoire	Prettier + CI
Variables	**const > let**	Préférer const. let uniquement si réassignation nécessaire. Jamais var.	ESLint prefer-const
Async	**try/catch**	Tout appel asynchrone (API, Prisma) enveloppé dans try/catch ou .catch()	ESLint @typescript-eslint/no-floating-promises

28.3 Politique de commentaires

Commentaires utiles — AUTORISÉS et ATTENDUS

- Règle métier non évidente : expliquer le pourquoi, pas le quoi
- Workaround documenté : décrire la contrainte contournée et le ticket associé
- JSDoc sur les fonctions exportées : @param, @returns, @throws

Exemple de bon commentaire :

```
// Vérification doublon AVANT insert : Prisma ne remonte pas d'erreur
// suffisamment descriptive sur la contrainte UNIQUE en PostgreSQL.
// Voir issue #42 pour le comportement observé.
```

Commentaires interdits

- Code commenté versionné : supprimer le code mort, Git conserve l'historique
- Commentaires décoratifs ou évidents : // incrémente le compteur — ne pas commenter
- TODO sans référence : // TODO: fix this — remplacer par // TODO(#123): fix this
- Commentaires mensongers : un commentaire qui ne correspond plus au code est pire que l'absence de commentaire

28.4 Zones complexes identifiées

Les zones suivantes du code GridLock requièrent une attention particulière et un commentaire explicatif :

- auth.middleware.js : vérification JWT + extraction du rôle — commenter les cas d'expiration et de token malformé
- participation controller — doublon check : expliquer pourquoi SELECT COUNT avant INSERT plutôt qu'un simple catch
- Prisma seed.js : l'ordre des insertions (User → Student → Event → Participation) est contraint par les FK — le documenter
- Intercepteur Axios : expliquer la logique de redirect différée pour éviter les boucles de login

28.5 Application aux productions IA

Conformément à US19 et US20, tout code produit par un agent IA doit :

- Respecter ces règles avant soumission
- Signaler explicitement les zones où il a pris un raccourci (dette technique)
- Ne jamais inclure de console.log en dehors des blocs de debug explicitement délimités
- Documenter ses propres TODO avec une référence à la US concernée